

Tower Induction and Interaction Trees: Experience Report

Refactoring a proof library, twice

Anonymous Author(s)

Abstract

Parameterized coinduction, or *paco* (Hur et al.), is a library for low-level coinductive reasoning in the Rocq Proof Assistant which greatly improves on Rocq’s awkward *cofix* tactic. But proofs using *paco* still involve intricate definitions and a wide array of library tactics that can make them long and complex. An alternative to *paco* is offered by Pous’s *rocq-coinduction* library, which mechanizes the chain construction and tower induction principle of Schäfer and Smolka. Tower induction promises coinductive proofs that rely on fewer tactics than *paco*, and unlike *paco* provides a uniform mechanism for proofs by coinduction and proofs of soundness of up-to techniques (Pous & Sangiorgi 2011, Schäfer & Smolka 2017).

We assessed that promise at scale by porting the coinductive-proof infrastructure of the Interaction Trees (ITrees) library, a large library that relies extensively on coinductive reasoning, from *paco* to *rocq-coinduction*. This first port was disappointing. Proof size went down, but tactic surface went up far more. Proof text fell 12%, from 400k to 352k characters, while the custom tactic code ITrees had to write in order to drive the library nearly tripled, a rise of 178%. In particular, *rocq-coinduction* required much client-side plumbing around its exposed tactics in order for them to function, and also contained several usability bugs that required client-side workarounds in multiple locations. Furthermore, development occasionally stalled because the underlying tactics in the tower-based library were written in OCaml and relied on complex under-the-hood invariants, which made them difficult to debug in practice.

We therefore rewrote the tactic layer of *rocq-coinduction* itself from scratch. The new implementation is Rocq-native, replacing the OCaml plugin with Gallina and Ltac, and gap: reduced it from the *paco* library to the *rocq-coinduction* library.

while providing more expressive tactics and removing the limitations above. Rerunning the ITrees port against it keeps that proof reduction and removes what it cost. Library tactic invocations fell 52% (2705 to 1307), the library defines 8 fewer wrapper tactics on top of enhanced coinduction compared to *paco* (20 to 12), and the build-time regression the first port introduced fell from 26% to 4%. These results suggest that tower-based coinduction is not only a theoretical simplification but a practical improvement over *paco* for large coinductive developments.

note. Two numbers changed since the talk, both in your favour, and one needs a decision. (a) “20 to 14” is now 20 to 12, eight fewer rather than six, because the shared *to_mon* driver collapsed two more families. Fixed above. (b) the *tacImplClaim* placeholder is a placeholder for the 65% claim, which depends on a counting convention you have not picked. Slide 83 says “reduced it by 65%”, from “–300 LOC of ocaml, –200 LOC of Rocq”. Counting like for like, comments and blanks removed on both sides, the tactic implementation goes 363 → 192 lines (–47%) or 12 233 → 6 164 characters (–50%); counting the whole of *tactics.v* against the plugin by *raw wc -l* gives –62%. Table 1 has the full breakdown. Pick one and I will wire it through the abstract, the macros and the checker.

1 Introduction

The paper from a 10,000 foot view:

We ported the **ITrees library**:

- < 16k LOC of Rocq
- < 330 proofs by **coinduction**

Why?

Hypothesis: Using *rocq-coinduction* will give shorter and more uniform proofs by coinduction than with *paco* (and using it requires fewer tactics) in ITrees.

note. Structure, from the talk. The talk has two major sections: *Context*, the theoretical foundations of coinduction and the evolution of those techniques in Rocq (Section 2–Section 4), and *Experiment Report* (Section 5–Section 9). This outline follows that order. The one structural difference is that the talk’s 44 context slides cannot become half the paper; see the note at the head of Section 2.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. CPP ’27,

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

note. Add a contributions list at the end of this section. CPP reviewers look for one, and the talk does not have a slide for it. The four the material supports: the refactoring itself; the negative result from the first port; the rewritten library; and the three-checkpoint quantitative evaluation.

2 Context: Coinduction as a Greatest Fix Point

note. This section compresses talk slides 6–28, which is the largest structural decision in the outline. The talk spends 44 slides on context. At CPP's 12-page limit that would consume half the paper and leave no room for the experiment, which is the contribution. The material below keeps the spine of Context 1 and Context 2 and drops the worked pictures. Everything cut is still in the talk, so an appendix or a pointer to the slides is an option.

2.1 The Need for Observational Equality

Idea 1: If all the observations τ or β within the process align, the two processes are (observationally) equal.

Idea 2: This can work even when the processes are *infinite*.

Two prominent examples of non-well-founded objects: Label Transition Systems (LTSs), often defined with Prop; and Infinite Objects (Processes, Streams...), defined with CoInductive, CoFixpoint. Infinite objects are the focus of this paper.

“Coinduction is used for writing an infinite proof about infinite objects”

Our core case study is coinduction for bisimulation, an instance of this framework. But unary coinductive predicates are very common: think properties for single programs!

Goal: need to be able to show objects “observe” the same *forever*. But we can't do this manually...

2.2 Theoretical Background

Coinduction in general admits models in terms of final coalgebras. But also order- and lattice-theoretic ones in terms of greatest fix points. This latter is simpler and sufficient for defining relations, and is the focus of this paper.

2.3 Lattices

A lattice is a set, a preorder defined on elements of the set, and supremum (least upper bound) and infimum (greatest lower bound) operators. We write this as $(X, \leq, \vee)$. (Greatest lower bounds can be derived from least upper bounds) w.l.o.g.

A *complete* lattice is a lattice in which **any subset of X admits a least upper bound and greatest lower bound**. A natural consequence of this is that every complete lattice has a maximum element (top, $\top$) and minimum element (bottom, $\perp$).

A function on a lattice is *monotone* if it respects the pre-order. That is, *mon* f means $\forall x y, x \leq y \rightarrow f x \leq f y$. A

fix point of a function f is an element x s.t. $f x = x$. **Every monotone function on a complete lattice admits a complete lattice of its fix points (Knaster–Tarski)**. This means every monotone function has a *greatest fix point* – [this will be important](#).

2.4 Focus: Lattices of Relations

These lattices exist because: (1) if X is a complete lattice, then $A \rightarrow X$ is a complete lattice; (2) Prop is a complete lattice, with $\leq \triangleq \rightarrow$.

$\leq$ on lattices of relations is determined *pointwise*: $R \leq S$ when $\text{forall } x, R x \leq S x$.

Our study: lattice of relations on states, $\text{state} \rightarrow \text{state} \rightarrow \text{Prop}$.

2.5 Why is this important for proving observational equivalence?

Consider a monotone function $b : \text{mon} (\text{state} \rightarrow \text{state} \rightarrow \text{Prop})$. b describes ‘[being observed and taking a step](#).’ In particular, $(b p q R)$ means p and q take the same step together, and are then related by R .

Consider a fix point of b : $(\text{fp } b) = b (\text{fp } b)$. If we relate two systems by $(\text{fp } b)$, they take the same b steps together, forever.

Fortunately... we know $(\text{fp } b)$ exists, for any monotone b . We know (at least two) such relations exist: the **least** and **greatest** fix points of any monotone function on our system. Our systems may run forever, so we pick the greatest fix point (gfp).

Then $\text{gfp } b$ is bisimilarity, which we will take to be observational equivalence.

2.6 The game: we want to find the greatest fix point of b

1. As a fix point, it unfolds infinitely: $\text{gfp } b = b (\text{gfp } b) = b (b (\text{gfp } b)) = \dots$
2. As the greatest fix point, it is inhabited for infinite judgements, while the least fix point is not.

There are various ways to construct $\text{gfp } b$ from b . Perhaps the most famous is a corollary of **Knaster–Tarski**: the lattice of fixpoints of b is complete, and its top is $\text{gfp } b$. Then the proof method to show membership in $\text{gfp } b$ is to show membership of anything under it.

Knaster Tarski: Game is to find R that satisfies this diagram. Or this definition: $R \leq bR$ (R is a post-fix point of b). Then you win: $R a b \rightarrow \text{gfp } b a b$.

gap. The proof principle, “Knaster–Tarski Coinduction”, as an inference rule (talk slides 27 and 28). The second form, $x \leq y \leq b y \Rightarrow x \leq \text{gfp } b$, is the one the rest of the paper uses.

3 Context: Mechanized Coinduction

But... why care about this when we have `cofix` in Rocq?

3.1 Problems with Rocq's Native Coinduction

1. Non-compositional
2. Easy to make mistakes with `cofix`
3. Guess the bisimulation candidate *up front*
4. Not amenable to automation

Another (big) problem:

5. No support for **up-to** techniques

3.2 Coinduction Up-to

Motivation: easier proof by coinduction.

$R \leq b \ R \longrightarrow R \leq b \ (f \ R)$ for some f

Example: $f \ x := x \ \backslash / \ \text{id}$. This is “**up to reflexivity**”. Could prove manually, but... would rather show $\sim$ is reflexive!

Other common up-to functions are symmetry, transitivity, and up-to context. Some up-to techniques, like up-to context, are essential for proofs in sophisticated developments.

3.3 (Mechanized) Coinduction Wishlist

1. Compositional (Non-syntactic guardedness)
2. Smooth way to prove soundness of **up-to techniques**
3. Straightforward to use
4. Amenable to automation
5. Nice-to-have: incremental bisimulation

How about just formalizing Knaster-Tarski?

gap. The scorecard from talk slide 35: 1 ✓, 2 ✗, 3 ✓, 4 ✓, 5 ✗.

note. This wishlist is the paper's spine and should be presented as such. The talk scores it four times, at slides 35 (Knaster-Tarski), 41 (paco), 64 (tower) and 89 (the rewrite, items 6–10). That repetition is what makes the talk's argument land, and it is a structure a paper can keep cheaply as one table with four columns rather than four separate scorecards. Worth deciding early, because it also fixes how Section 9 ends.

3.4 The Pursuit of Happiness in Coinduction

gap. The timeline from talk slide 37: `cofix` → `paco` → the companion (Pous [3]; Parrow & Weber; Schäfer & Smolka [4]) → ITrees and `gpaco` → New ITrees. This is the one context figure that earns a full column in the paper, because it places the two libraries the paper compares.

4 Context: `paco`, `gpaco`, and Interaction Trees

4.1 Improvement 1: `paco`/`gpaco`

`paco` [2] (parametrized coinduction) formalizes Knaster-Tarski...allowing us to find the `gfp` in a more permissive way...by giving us rules for accumulation and composition.

gap. $G_f(x) \stackrel{\text{def}}{=} \nu y. f(x \sqcup y)$, with G the “parameterized” `gfp`, and $\nu f \equiv G_f(\perp)$. Then the three rules from slide 39: COMPOSE, for

combining theorems; ACCUMULATE, for adding knowledge during a proof; UNFOLD, for exposing the monotone function f .

`gpaco` [6] (generalized **paco**) exploits *compatible enhancements* to the `gfp` for **up-to** reasoning.

Paco stats:

gap. The scorecard from slide 41: 1 ✓, 2 (partial), 3 ✓, 4 ✓, 5 ✓...but much better than `cofix`!

4.2 From `paco`, Interaction Trees (ITrees) [5]

Idea: Abstract away all coinductive proof, and do reasoning modulo an equational theory.

ITree: A data structure for:

1. Modeling infinite and impure computation
2. Hiding the low-level coinductive reasoning one would do with `paco`.
3. Extractable execution for verified software

`itree` is the CoInductive `itreeF`.

`Paco` is pervasive in ITrees, and is used to define bisimilarity.

gap. The search hit from slide 44, “667 results in 54 files”, beside `eqi tF` as the `b` and `eqi t` as the `gfp b`. This is the slide that justifies calling the port large.

4.3 ...but `paco` has some weaknesses

- Long compatibility proofs
- Lemmas reasoning about compatibility are hard to read and maintain
- Lots of individual tactics make it a bit tricky to use and automate (6 `paco`-specific tactics in this proof)

gap. The three code figures from slides 46, 47 and 48. At least one has to survive into the paper; the 6-tactics-in-one-proof example on slide 48 is the one that carries a number.

4.4 Can we do better?

`Paco` is a complete theory of up-to coinduction: Problem is only in usability.

Main problems:

1. Compatibility proofs can get long,
2. Proofs with `gpaco` closures can get confusing and use different techniques,
3. Lots of tactics to remember (4-8+ unique `paco` tactics per proof)
4. Minor wart: “arity hell:” each `paco` operator is re-defined for different arities, e.g., `paco1`, `paco2`, up to `paco16`... (also makes portable automation harder to write)

So, our goal is to find a theory that gives “nicer” proofs, is easier to maintain and teach, etc.

5 Attempt 1: Tower Induction in the rocq-coinduction library [3, 4]

5.1 The Tower

“Start at the top and b down to the gfp”

Inductive C: $X \rightarrow \text{Prop} :=$
 | Cb: $\forall x, C x \rightarrow C (b x)$
 | Cinf: $\forall T, T \leq C \rightarrow C (\text{inf } T)$.

The tower is closed under b. The tower is closed under infima (greatest lower bounds).

Transfiniteness recovered: $\text{gfp } b = \text{inf } C = b^\lambda(\top)$.

gap. The unrolling from slides 56–58: $\text{inf } \backslash\{\backslash\} = \top$, then $b \top, b^2 \top, b^3 \top, \dots, \text{inf } (b^n \top)$, converging on $\text{inf } C = \text{gfp } b$. Slide 58 also carries an open problem worth stating in the paper: *prove this is the same as Parrow & Weber’s ordinal construction.*

5.2 The Tower: The best part

We get an inductive proof principle to do coinduction! And indeed to do more...

5.3 Up-to and coinductive proofs with the tower

To prove f is a sound up-to of the tower, **prove f holds of every element of C by induction on C.**

This *tower induction* technique is the single proof method for both up-to proofs and coinduction.

Indeed, to prove two elements are related by the gfp, you can use tower induction:

1. Turn the relation into a predicate P
2. Prove P holds for all elements of the tower by tower induction
3. The gfp is in the tower, so P holds for the gfp.

This can be done automatically. The base case is almost always trivial.

If we have no base case, we just have to prove stability under b... This looks just like coinduction!

gap. “I can’t believe it’s not coinduction!” – the goal state from slide 63, where the tower-induction hypothesis is literally the coinduction hypothesis. This is the payoff of the whole context section and should be a figure.

5.4 Tower stats (my claim)

gap. The scorecard from slide 64, all five green: 1. Compositional; 2. Non-syntactic guardedness; 3. Nice-to-have: incremental bisimulation; 4. Amenable to automation (very few tactics, all with uniform behavior); 5. Smooth way to prove soundness of **up-to techniques** – (also) by *induction*!

6 Theory meets Practice: porting ITrees from paco to the tower

Improvements Anticipated... Improvements Found?

Under PACO

Lemma $\text{eqit_inv_Tau } b1 \ b2 \ t1 \ t2 :$
 $\text{eqit } RR \ b1 \ b2 \ (\text{Tau } t1) \ (\text{Tau } t2) \rightarrow \text{eqit } RR \ b1 \ b2 \ t1 \ t2.$
Proof with eauto with itree.
 intros. punfold H. red in H. simpl in *.
 remember (TauF t1) as tt1. remember (TauF t2) as tt2.
 hinduction H before b2; intros; try discriminate.
 - inv Heqtt1. inv Heqtt2. pclearbot. eauto.
 - inv Heqtt1. inv H.
 + pclearbot. punfold REL. pstep. red. simpobs...
 + pstep. red. simpobs. econstructor; eauto.
 pstep_reverse. apply IHeqitF; eauto.
 + eauto with itree.
 - inv Heqtt2. inv H.
 + pclearbot. punfold REL. pstep. red. simpobs...
 + eauto with itree.
 + pstep. red. simpobs. econstructor; auto.
 pstep_reverse. apply IHeqitF; eauto.
Qed.

Under enhanced coinduction

Lemma $\text{eqit_inv_Tau } t1 \ t2 :$
 $\text{eqit } RR \ b1 \ b2 \ (\text{Tau } t1) \ (\text{Tau } t2) \rightarrow \text{eqit } RR \ b1 \ b2 \ t1 \ t2.$
Proof.
 intros.
 step in H; step.
 now apply eqitF_inv_Tau.
Qed.

Figure 1. The same lemma at the paco and new port checkpoints. The statement is identical; only the proof differs. The PACO proof interleaves coinductive plumbing with an induction on the derivation, and six of its tactic calls (punfold, pclearbot, pstep, pstep_reverse) exist only to move between PACO’s guarded and unguarded forms. The port separates the two. Taking one step in the hypothesis and one in the goal is `step in H; step`, and the induction becomes `eqitF_inv_Tau`, an ordinary 19-line lemma about the one-step functor that contains no coinduction at all and is reused elsewhere. The total is not much smaller. What changes is that the coinductive reasoning is three lines and is separable from everything else.

Was this experiment a success? What did it tell us?

6.1 Initial Results are good

Up-to proofs got shorter.

Broadly, about a 10-12% decrease in proof and definition verbosity.

Best recorded improvement: 1,490 words 9,725 characters $\rightarrow$ 570 words 4,391 characters.

gap. The side-by-side from slide 67, paco 407 words 2,610 characters against tower 258 words 1,364 characters.

note. “proof and definition verbosity” is half right. Proof fell 8.7% in theories/ and 12.0% across the whole development at this checkpoint, so 10–12% is fair for proof. Definitions did not move: –0.2% and +0.3%. Table 2 has both. Either drop “and definition” or say proof fell 10–12% while definitions stayed flat, which is a cleaner setup for Section 7 because it isolates where the cost went.

6.2 Initial Results, pt. 2

Many proofs by coinduction got shorter. 209 words 1,281 characters → 117 words 671 characters.

They also use fewer library tactics: This example: 7 vs 2.

6.3 Other informal observations

- Proofs by coinduction and up-to proofs look very similar
- We depend on a smaller library that builds 5x faster
- Proofs are easier for me to read and explain
- LOC # is theoretically* much lower
- No need to remember arities, everything is gfp

note. Two of these are unmeasured and a reviewer will ask. “builds 5x faster” is not in the released data, and “LOC # is theoretically much lower” has an asterisk in the talk with no footnote. Either measure them or mark them explicitly as informal, which the section heading already does.

7 First-pass problems

But... some features the library was missing:

6. No bugs ×
7. Informative error messages ×
8. Rich tactic support ×
9. Good performance (strange but important in some cases) ×
10. Documentation of all points thus far (Some)

note. These continue the numbering of the wishlist in Section 3.3, which is the talk’s device for showing that the first five criteria were met and the development still went badly. Keep the numbering; it is what makes Section 9’s rundown legible.

Tactic surface in ITrees up 178%. Needed 20 custom tactics (vs 14 from paco).

note. The tactic counts do not match any current measure and need replacing. Measured, the count of distinct coinduction-driving tactic names goes paco → old port: 16 → 21 in theories/, and 20 → 23 across the whole development. So the first port needed 23 against paco’s 20, not 20 against 14. The 178% is correct and is the whole-development character figure. My guess is that “20 to 14” was paco against the *new* port at an earlier measurement, which is now 20 → 12; that belongs in Section 9, not here.

Why?

1. rocq-coinduction tactics **not expressive enough**
2. Some rocq-coinduction tactics were **buggy**

Unusable in certain cases, which required extensive workarounds. Error messages could get very confusing.

7.1 Confusing errors (& bugs)

gap. The two error figures from slides 75 and 76. Both are reproducible and released: see paper-data/old-library-errors/. The second is the stronger one, and it is the bug:

```

V (t1 : itree E R1) (t2 : itree E R2),
gfp (eqit_mon b1 b2) R1 R2 RR t1 t2 →
gfp (eqit_mon b1 b2) R1 R2 RR' t1 t2

```

coinduction R CIH on this goal reports No such section variable or assumption: R. The message names R, the candidate the user just wrote, so it reads as though the user’s script is at fault rather than the goal shape being unsupported.

7.2 Why debugging was hard

Debugging my own code was **hard**. Fixing the bugs was **also hard**.

Why? rocq-coinduction uses a reifying OCaml plugin with a complex Rocq invariant.

gap. The two quotes from slide 79, which are the sharpest evidence in the talk that the plugin was the obstacle: “the key invariant is that $[e]$ should be convertible to $[pT a c r x]$ ” and “the OCaml type of $[g]$ is a bit complicated: $[bool \rightarrow int \rightarrow (int \rightarrow constr) \rightarrow constr]$ ”.

8 Solution: bypass reification

Rewrite the rocq-coinduction library tactic interface

Lightning Overview:

- Removed OCaml to use pattern
- Prove inf-closedness using icauto
- Proved monotonicity using monauto

8.1 Improvement

Old:

```

Tactic Notation "coinduction" ident(R) simple_intropattern(
  H) :=
  apply gfp_prop; intro R; apply_ptower R 0; intros H.

```

apply_ptower R 0: ~200 Lines of OCaml + ~200 Lines of Rocq.

New:

```

Tactic Notation "coinduction" ident(R) simple_intropattern(
  H) :=
  gfp_intro' R; pattern (elem R); revert R;
  apply tower; [ icauto | intro R; cbn beta; intros H ].

```

Total: ~25 lines of Rocq.

8.2 Tradeoff

- Removed OCaml to use pattern
- **Tradeoff: Prove inf-closedness using icauto**

Table 1. The coinduction library before and after the rewrite, in comment-free non-blank lines. The tactic implementation is the tactic file together with the OCaml plugin it called. The plugin is gone, so the rewritten library has no OCaml in it at all.

Component	Released	Rewritten	Change
<i>Tactic implementation</i>			
tactics.v	229	188	−18%
OCaml plugin	307	0	−100%
total	536	188	−65%
<i>Rest of the library</i>			
lattice theory	384	499	+30%
tower	134	150	+12%
companion	523	499	−5%
relations	124	148	+19%
Core total (excluding tests)	1 395	1 485	+6%
Comments	370	455	+23%
Tests	84	328	+290%

```

Ltac icauto :=
  tryif solve [auto 20 with ic mon nocore] then idtac
  else match goal with
    | |- inf_closed ?P =>
      fail 1 "[coinduction] cannot show that this
      predicate is inf-closed:" P
    | _ => fail 1 "bug in icauto, please report"
  end.

```

Nondeterministic, but adjustable by users with auto depth.

8.3 Attempt 2: With fixed library

Rewrite the rocq-coinduction library tactic interface

Removed ocaml plugin entirely, which simplified the tactic codebase and reduced it by `gap`: reduced it by `[pick a convention]`. In particular: −300 LOC of ocaml, −200 LOC of Rocq. Added support for more expressive tactics that didn't require so much wrapping. Fixed the bugs.

note. The −300/−200 split holds up; the headline percentage depends on convention. Table 1 measures every line that exists to implement tactics, comments and blanks removed on both sides. The OCaml plugin is 174 lines (307 by raw `wc -l`, which is where −300 comes from) and it needed a further 139 lines of Rocq glue — the Module reification block, 17 Register declarations and the Declare ML Module — plus 7 lines of build config, which is the −200. Against that, Ltac grew 43 → 192 lines, because the logic moved out of OCaml and into Ltac. Totals: 363 → 192 lines, −47%; 12 233 → 6 164 characters, −50%. See the abstract note for the four conventions and pick one.

9 Results

9.1 Tactics

Tactic surface in ITrees down 45.4%

9.2 Proofs

Proof reduction largely maintained

9.3 Definitions

Definitions reduced slightly

9.4 Total

Gains across the board, with tactic & proof gains significant

note. Every number on talk slides 84–87 reproduces exactly against the current data, because those slides use the released tables. Table 2 is the same table with total size added. The four figures are in slides/out/ as tactic, proof, def and total.

note. One number the talk does not have, and should. Total size, every non-blank comment-free line counted once, goes 20 871 → 20 455 → 20 156 across the whole development, which is −2.0% at the first port and −3.4% after the rewrite. It never rose. That is worth stating because “the tactic surface tripled” invites the reader to assume the development grew, and it did not.

9.5 Other perks

icauto is user-facing, previously internal only.

monauto is user facing, previously nonexistent (but was in paco).

gap. The two before/after pairs from slide 88. The first collapses six lines of `apply inf_closed_* plumbing` into `tower induction.`; the second collapses a five-line monotonicity proof into `Proof. monauto. Qed.` Supporting figure: monauto discharges 17 of the 18 monotonicity statements in ITrees, and tower induction is used at 22 sites while ITrees contains zero calls to icauto, because the obligation never reaches the user.

9.6 Rundown

Old → new port:

6. No bugs $\times \rightarrow \checkmark$
7. Informative error messages $\times \rightarrow \checkmark$
8. Rich tactic support $\times \rightarrow \checkmark$
9. Good performance (strange but important in some cases) $\times \rightarrow \checkmark$
10. Documentation of all points thus far (Some) → (More) now

note. Item 9 now has a number behind it. The first port cost +25.6% on a full serial build against the paco baseline; the rewrite recovers it to +4.1%. Four interleaved rounds, protocol in slides/README.md. Diagnosed causes, none inherent to enhanced coinduction: the port

Table 2. The three checkpoints on every measure. *Tactics* counts distinct coinduction-driving tactic names; *lines* and *chars* give the size of their definitions, every copy counted. Definition figures include tactic definitions, since `L tac` is specification for the classifier. *main* is the library proper; *main+extra* adds the four downstream developments.

Checkpoint	Scope	Coinduction tactics			Definitions		Proofs	
		names	lines	chars	lines	chars	lines	chars
paco	main	16	118	4 181	5 870	213 716	6 231	176 031
paco	main+extra	20	127	4 586	8 853	354 025	12 018	399 700
old port	main	21	240	8 537	5 820	213 284	6 119	160 788
old port	main+extra	23	360	12 754	8 934	355 226	11 521	351 873
new port	main	11	66	2 283	5 613	206 132	6 133	161 374
new port	main+extra	12	71	2 545	8 625	344 902	11 531	352 794

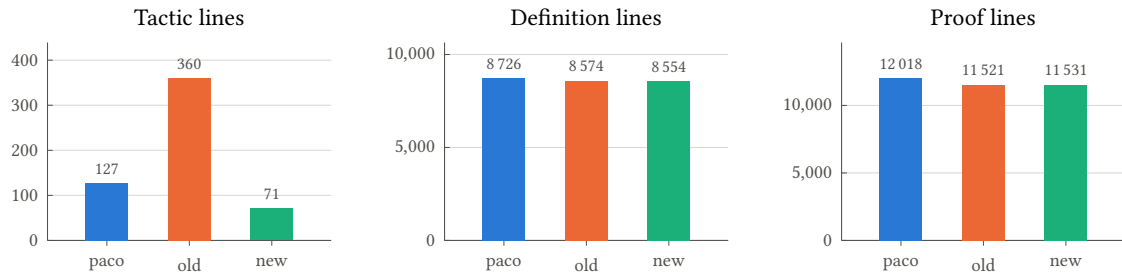


Figure 2. The whole development at the three checkpoints, counting the library proper and its four downstream users together. Each panel carries its own scale, because the three measures differ by two orders of magnitude, and every panel is zero-based. Definition lines exclude tactic definitions, which the first panel counts. Definitions and proofs barely move across the three checkpoints. Tactic code, the measure enhanced coinduction was expected to reduce, almost triples at the first port and returns below the PACO baseline only after the library was rewritten.

Table 3. Wall-clock time for a full serial build of Interaction Trees at each checkpoint, in seconds. Builds are interleaved rather than blocked, and the coinduction library is prebuilt, so the timed region is the Interaction Trees development itself. Mean over 4 rounds, and over the 3 rounds after the cold first one.

Checkpoint	All rounds		Warm rounds	
	mean	vs. PACO	mean	vs. PACO
paco	85.4	—	84.7	—
old port	106.9	+25.2%	106.4	+25.6%
new port	88.9	+4.1%	88.2	+4.1%

dropped `paco`'s explicit `eauto` depth annotations; `inv` gained a try easy and is called at 1118 sites; and 33 sites reverted the whole context instead of the tail.

Those fixes are also why proof text is 0.3% *longer* at the new port than at the old one: `eauto 4` with `itree`, `inv_e` and `try discriminate` are each a few characters longer than what they replaced, across several hundred sites. Worth one sentence, since Table 2 shows the proof column ticking up and a reviewer will ask.

10 Ideas and Future Work

1. Main Ambition: even better up-to with diacritical companions

Imagine a proof by coinduction of *weak* bisimilarity (bisimilarity modulo invisible steps). We can rewrite by **strong bisimilarity**, but **not by weak bisimilarity** in general. There are special cases where we can, though! How? Only if we use CIH under beta, not tau. `gpaco` lets us do this, and so does the *diacritical companion* [1], a newer discovery. Can we recover this power in the land of the tower?

There exists modern work which could recover some of the power of `gpaco` within the tower framework, but if this is usable remains to be seen (part of my summer work at Inria).

2. A write-up on the tower in the library

Because history is nonlinear, no write-up of the enhanced coinduction library as it exists.

3. Immediate: Upstream to ITrees/rocq-coinduction

Usability of the enhanced coinduction library is good, but there are bugs. Also, there are typical design patterns with the library that probably should be part of it.

note. Item 2 is a contribution the paper can partly claim rather than defer: the rewritten library ships with its test suite grown 84 $\rightarrow$ 328 lines and its comments 370 $\rightarrow$ 455. Item 3's "design patterns that should be part of it" now has a worked instance, the shared `to_mon` driver, which removed the per-functor blocks from three downstream files and took extra/'s tactic cost from 895 characters below `paco`'s 405, to 262.

Data-Availability Statement

note. Not in the outline; added because CPP has an artifact track and reviewers look for this. Draft text below, to keep or cut.

The rewritten coinduction library, both ports of Interaction Trees, the measurement script, the raw data behind every table in this paper, and the file of behavioural claims stated as compiling Rocq goals are available as supplementary material.

gap. Archival DOI for the camera-ready.

References

- [1] Dariusz Biernacki, Sergueï Lenglet, and Piotr Polesiuk. 2019. Diacritical Companions. *Electronic Notes in Theoretical Computer Science* 347 (2019), 25–43. doi:10.1016/j.entcs.2019.09.002 Proceedings of MFPS XXXV.
- [2] Chung-Kil Hur, Georg Neis, Derek Dreyer, and Viktor Vafeiadis. 2013. The power of parameterization in coinductive proof. In *Proceedings of the 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. ACM, 193–206. doi:10.1145/2429069.2429093
- [3] Damien Pous. 2016. Coinduction All the Way Up. In *Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*. ACM, 307–316. doi:10.1145/2933575.2934564
- [4] Steven Schäfer and Gert Smolka. 2017. Tower Induction and Up-to Techniques for CCS with Fixed Points. In *Relational and Algebraic Methods in Computer Science (RAMiCS) (Lecture Notes in Computer Science, Vol. 10226)*. Springer, 274–289. doi:10.1007/978-3-319-57418-9_17
- [5] Li-yao Xia, Yannick Zakowski, Paul He, Chung-Kil Hur, Gregory Malecha, Benjamin C. Pierce, and Steve Zdancewic. 2020. Interaction Trees: Representing Recursive and Impure Programs in Coq. *Proceedings of the ACM on Programming Languages* 4, POPL, Article 51 (2020). doi:10.1145/3371119
- [6] Yannick Zakowski, Paul He, Chung-Kil Hur, and Steve Zdancewic. 2020. An Equational Theory for Weak Bisimulation via Generalized Parameterized Coinduction. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*. ACM, 71–84. doi:10.1145/3372885.3373813